ISPW

2025-2026

University of Rome Tor Vergata.

Lorenzo Brondi,
Simone Remoli.

1. SOFTWARE REQUIREMENT SPECIFICATION

1.1 Introduction

1.1.1 Aim of the document

The purpose of this document is to present the fundamental and specific requirements of the "*RouteX"* system, a project developed for the Software Engineering and Web Design course, Academic Year 2025–2026.

The document provides a comprehensive overview of the system's objectives, core functionalities, technical requirements, and the rationale behind the design choices, in order to guide the development process and ensure alignment among all team members.

Prepared by: Lorenzo Brondi and Simone Remoli.

1.1.2 Overview of the defined system

The system is a Java-based software platform structured according to the MVC architecture, developed using Maven, and featuring a graphical user interface implemented through JSP files.
The system was not developed using JavaFX and FXML files. Instead, JSP pages were adopted for the graphical layer. These pages are solely responsible for data presentation and therefore do not contain any application or business logic.

The system supports the efficient and modern management of urban routes within a given city, including the following functionalities:

- Management of travelers, workers, and administrators.
- Management of metropolitan routes across multiple cities.
- Ticket purchase and activation, with validation performed through QR codes.
- Creation and handling of reports related to stations, aimed at maintaining safety, monitoring service disruptions, and promptly addressing operational issues.
- Viewing reports and statistical data to support administrative monitoring, performance analysis, and decision-making processes.
- Shortest path computation between two stations.

There are three user types with different functionalities: Travelers, Workers and Administrators

1.1.3 HW and SW requirements

Software requirements:

- Java 17+
- Draw.io (for creating UML diagrams)
- IDE: Intellij IDEA or Eclipse
- MySQL Server
- Apache Tomcat 9.0.96 (web server)

Hardware requirements:

- CPU: Dual-Core 2.0 GHz
- RAM: 2 GB
- Disk Space: 1 GB
- Operating System: Windows, MacOS, Linux (compatible with java 17+)

1.1.4 Related systems

System 1: Moovit

Pros:

- Excellent and user-friendly mobile interface.
- Real-time public transport updates and service alerts.
- Wide coverage across multiple cities worldwide.

Cons:

- Limited support for role-based user management (e.g., workers and administrators).
- No integrated ticket purchasing and activation via QR code in most regions.

System 2: Google Maps

Pros:

- Accurate shortest-path computation between stations.
- Strong integration with real-time traffic and transport data.
- Highly reliable and scalable infrastructure.

Cons:

- Not specifically designed for metropolitan transport systems.
- Lacks administrative tools for route management and reporting.
- No native ticket purchasing or QR-based validation features.

System 3: Citymapper

Pros:

- Highly optimized routing algorithms specifically tailored for urban public transportation systems.
- Clear and intuitive visualization of routes, transfers, estimated travel times, and walking segments.

Cons:

- Lacks tools for internal system management, including route configuration, station monitoring, or maintenance reporting.
- Not very customizable for small businesses.

RouteX aims to combine route computation, ticket management, and role-based system administration within a single platform, providing dedicated functionalities for travelers, workers, and administrators, while maintaining a modular and extensible architecture

1.2 User Stories

Traveler:

- As a traveler, I want to find the shortest route between two metro stations, so that I can reach my destination in the most efficient way.
- As a traveler, I want to purchase metro tickets, so that I can use them to travel.

Worker:

- As a worker, I want to read all the maintenance reports related to the stations, so that I can promptly identify issues and take action to fix them.
- As a worker, I want to consult my work schedule, so that I can keep track of my assigned shifts.

Admin:

- As an administrator, I want to check reports and statistics about completed trips, so that I can analyze overall trends and make informed decisions.
- As an administrator, I want to create maintenance notifications, so that I can communicate maintenance-related updates and service interruptions.

1.3 Functional Requirements

Shortest Path:

- The system shall compute and display the shortest path between two metropolitan stations.

Maintenance report:

- The system shall create and display reports regarding the status of the stations.
- The system shall store a report as completed when it is marked.

Purchase Ticket:

- The system shall provide the purchase of one or more metro tickets for the selected city.

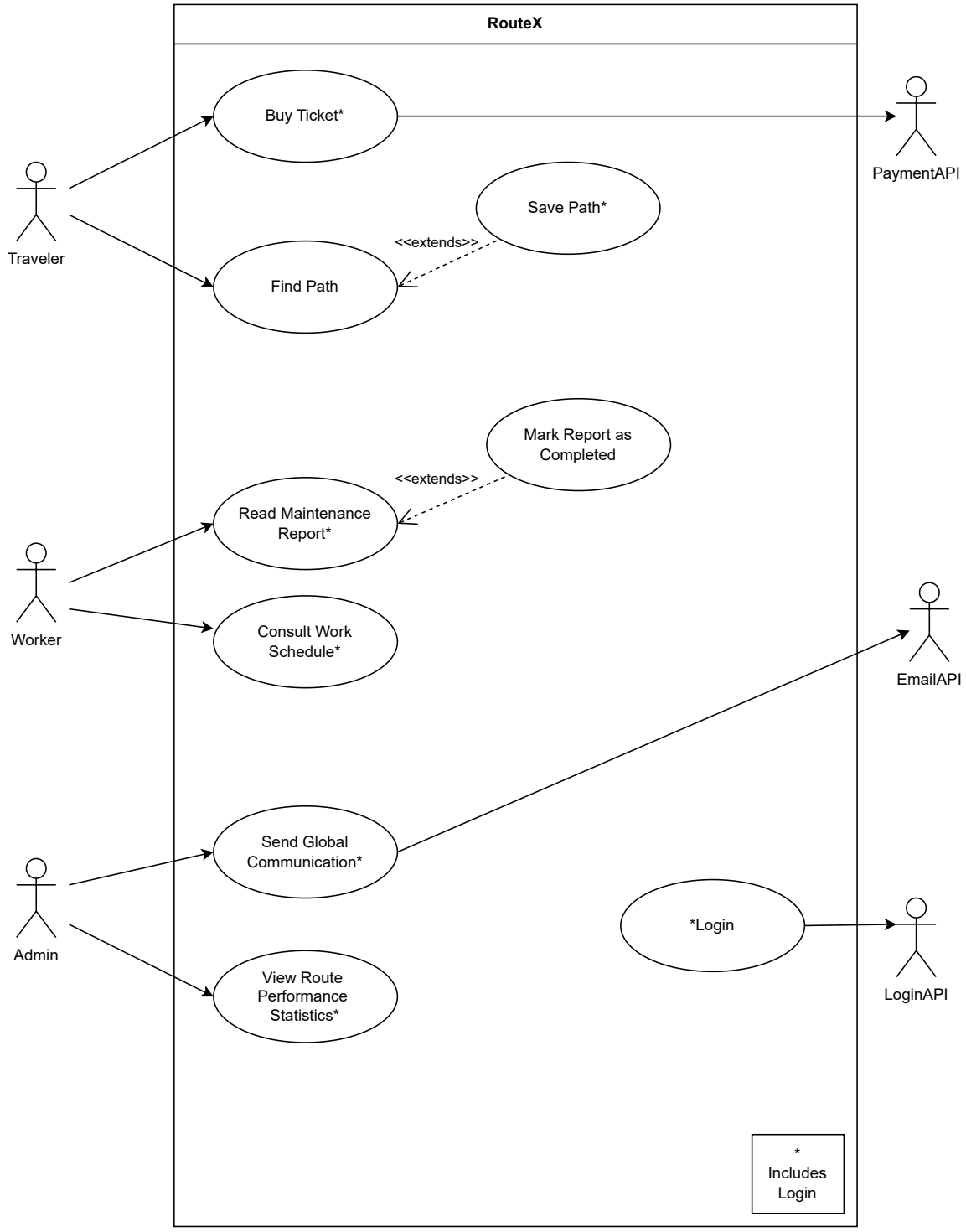
Access Code:

- The system shall generate a unique access code associated with a purchased metro ticket.

Statistics:

- The system shall generate and display statistics related to routes' performance.

1.4 Use Cases Diagram



1.4.1 Internal Steps

Use Case: Buy Ticket

Main flow:

1. The traveler **accesses** the "Buy Ticket" section.
2. The system **displays** the available destinations.
3. The traveler **selects** the desired destination.
4. The system **verifies** the destination.
5. The traveler **chooses** the number of tickets to buy.
6. The system **calculates** the total price to be paid.
7. The traveler **selects** the payment method.
8. The system **processes** the payment, notifies the traveler, generates and activates a ticket code.

Extensions:

4a. **The traveler specifies an unavailable city:** The System displays an error message indicating that the selected destination is not available.

5a. **The traveler selects an invalid number of tickets** (**e.g., exceeding the maximum allowed**): The System notifies the Traveler of the invalid selection, disables the purchase option, and requests to enter a valid number of tickets.

8a. **The payment fails:** The system notifies the traveler of the failed transaction and allows them to retry the payment process.

Use Case: Send Global Communication

Main Flow:

1. The admin **accesses** the "Send Global Communication" section.
2. The system **prepares** and **displays** a blank message form.
3. The admin **writes** the message and selects the "Send Message" button.
4. The system **checks** whether the number of characters has not exceeded the allowed limit.
5. The system **asks** to confirm the sending of the message.
6. The admin **selects** "Confirm".
7. The system **checks** the validity of the message.
8. The system **displays** a confirmation indicating that the message has been successfully sent.

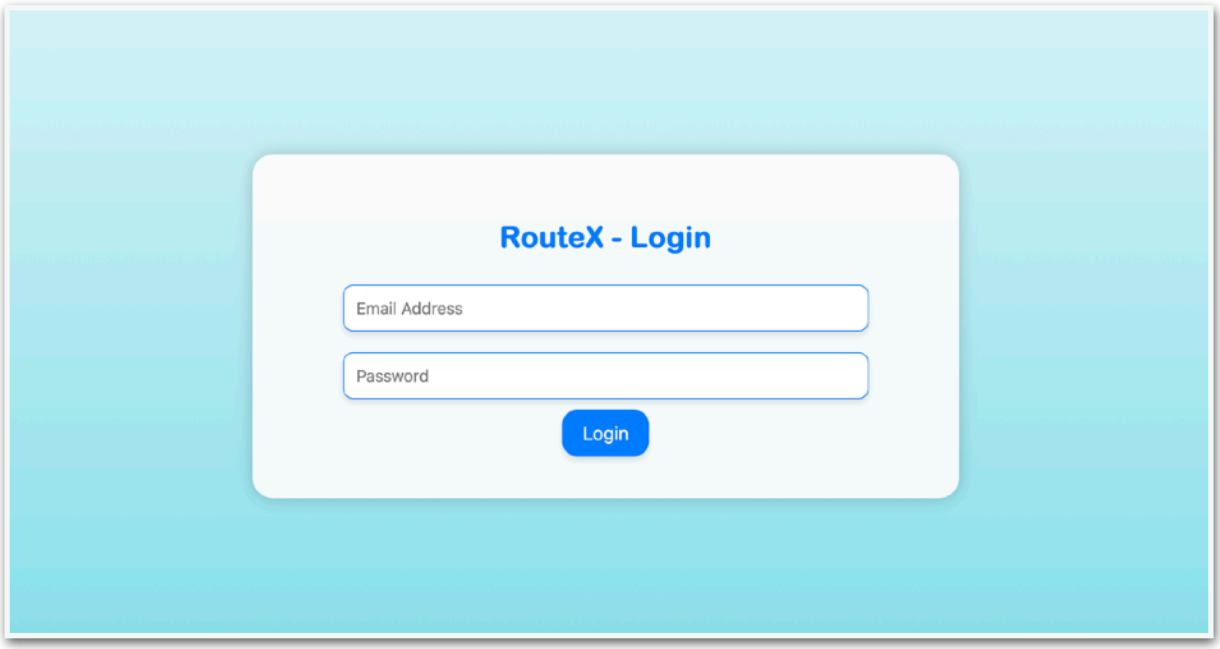
Extensions:

4a. **The admin exceeds the character limit**: The system displays a warning indicating that the maximum allowed length has been exceeded, and terminates the use case.

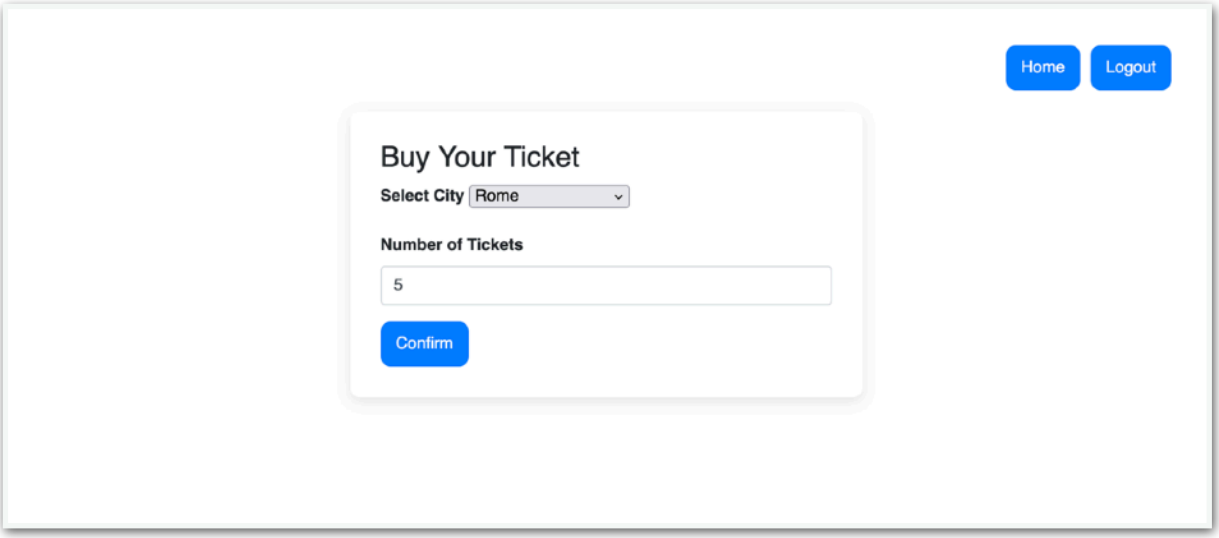
7a. **The message sending fails** (**e.g., offensive language or blasphemy**)**:** The system notifies the admin of the failure and allows them to modify and resend the message.

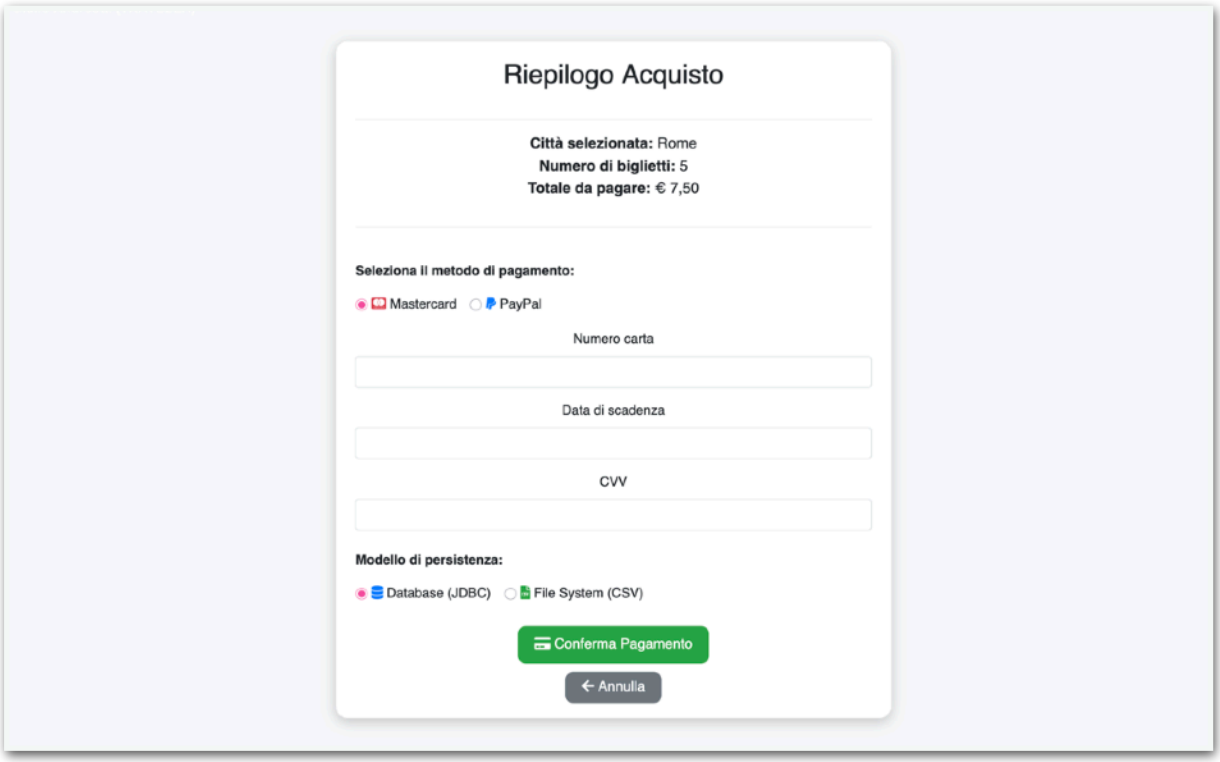
2. STORYBOARDS

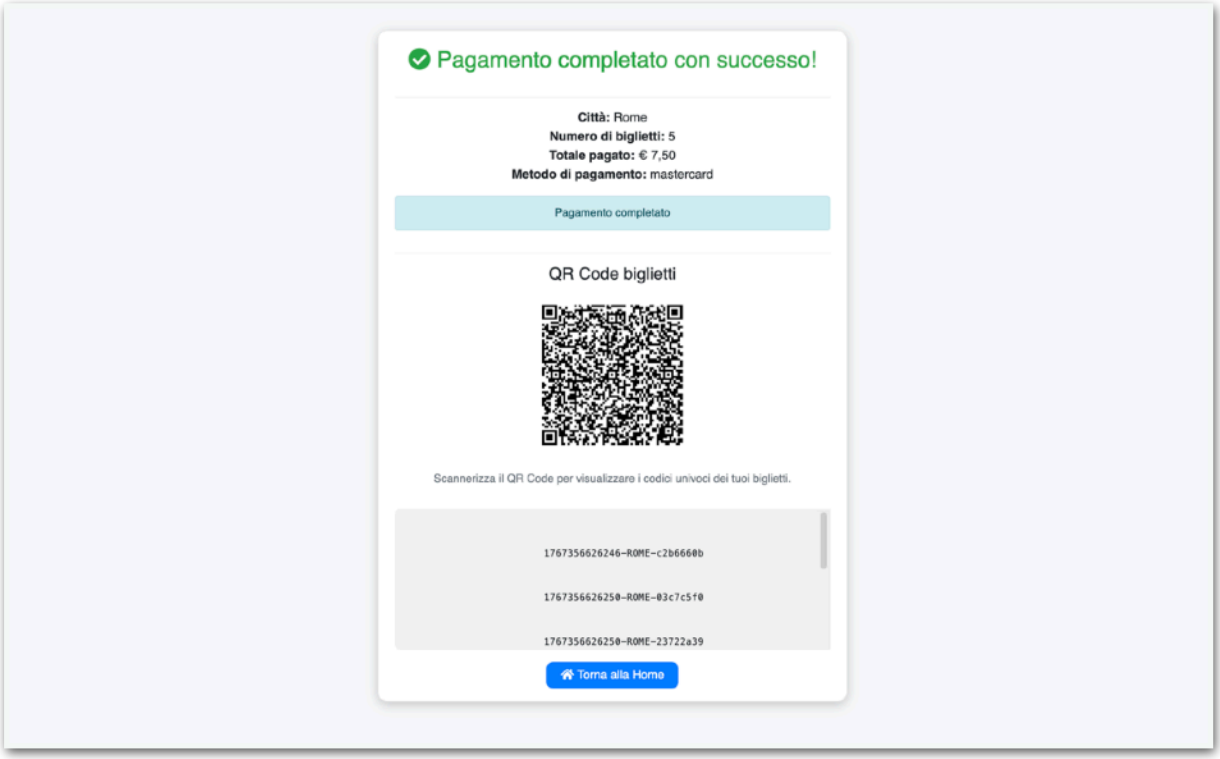
Login:



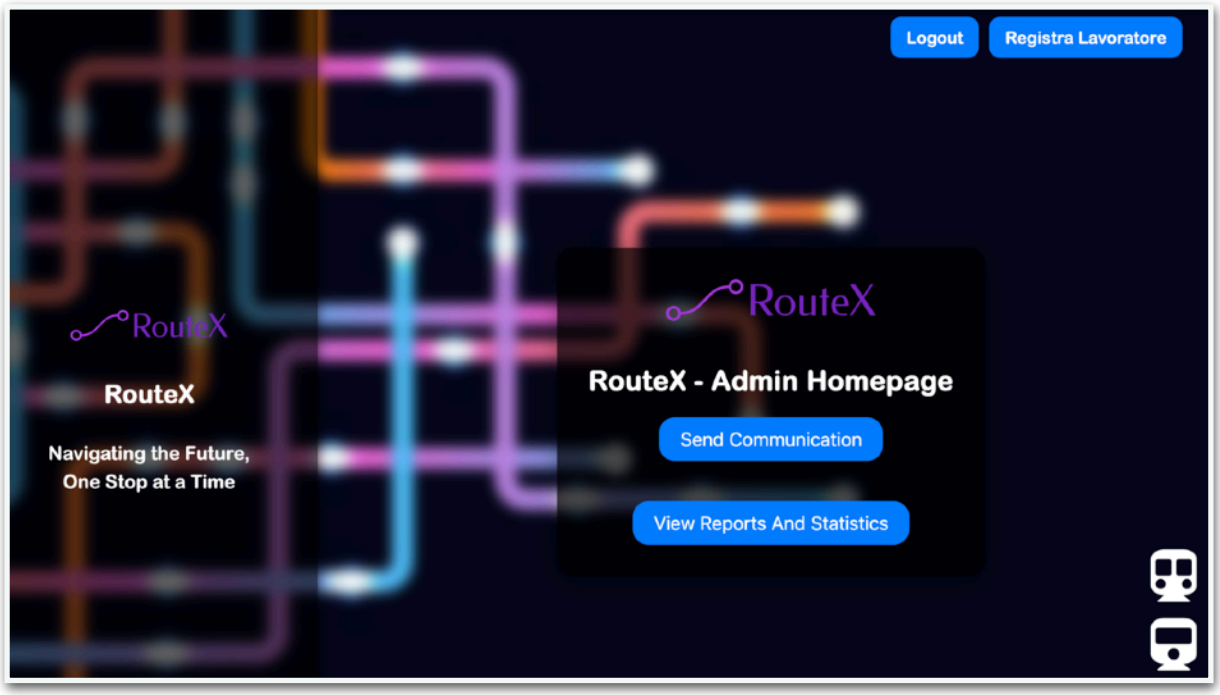
Buy Ticket:

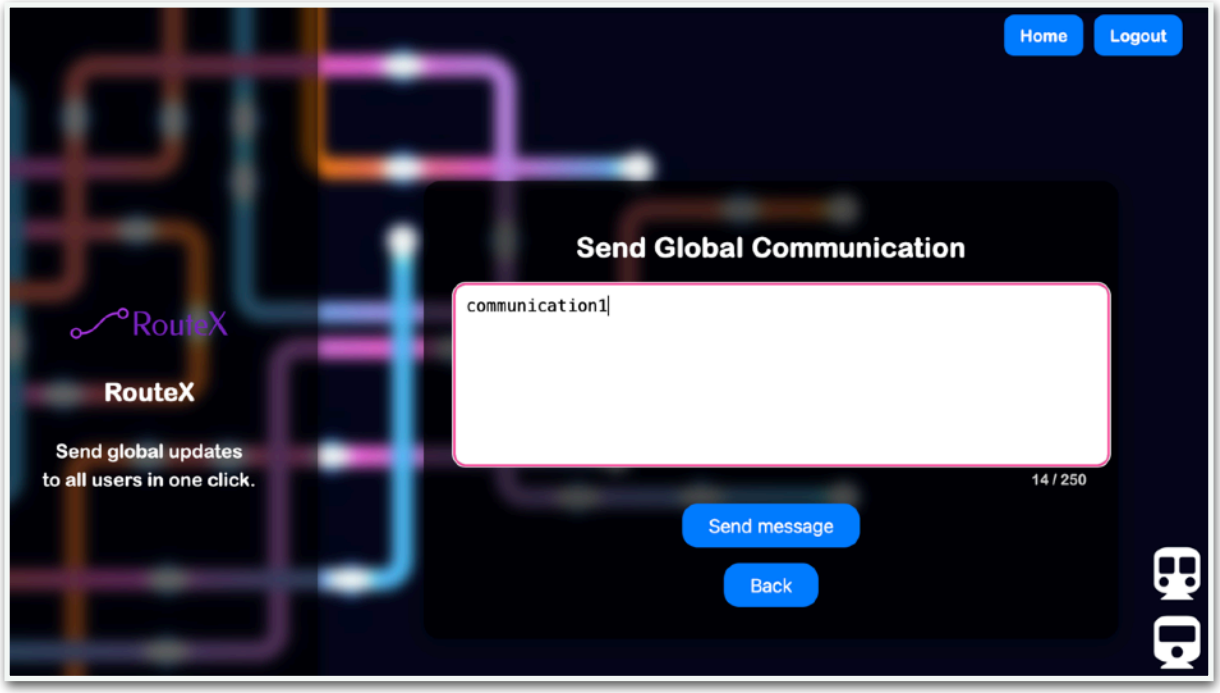


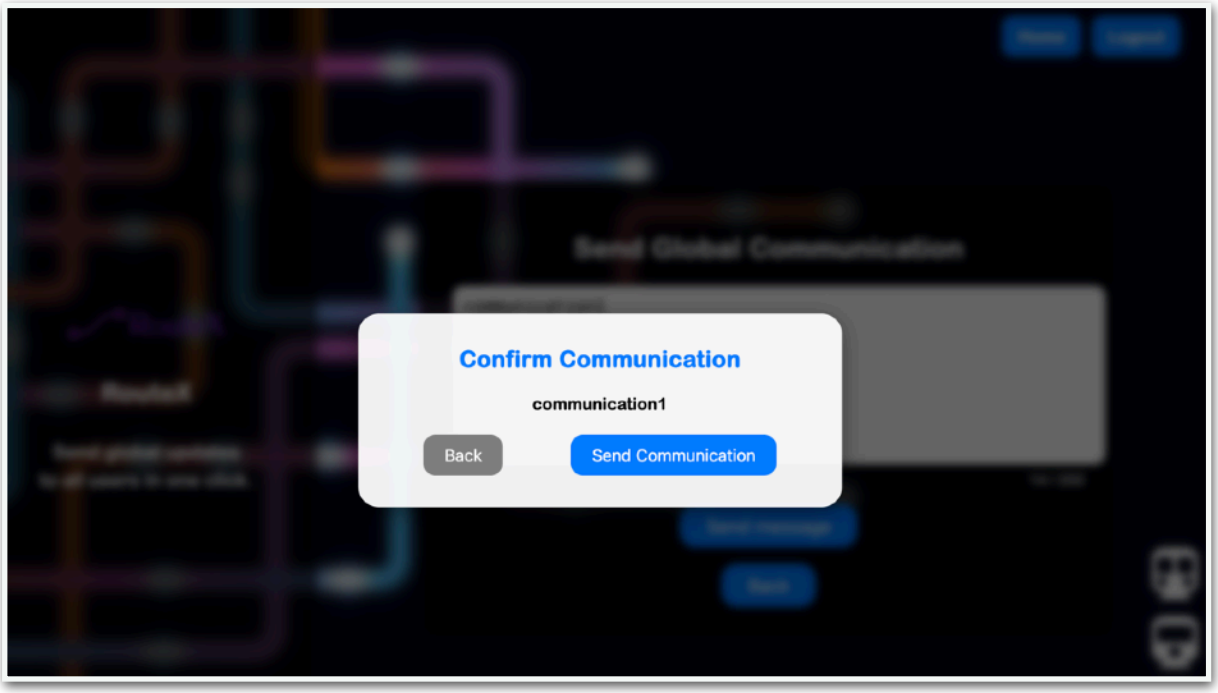

Riepilogo Acquisto
Città selezionata: Rome
Numero di biglietti: 5
Totale da pagare: € 7,50
Seleziona il metodo di pagamento:
Mastercard
PayPal
Numero carta
Data di scadenza
CVV
Modello di persistenza:
Database (JDBC)
File System (CSV)
Conferma Pagamento
Annulla

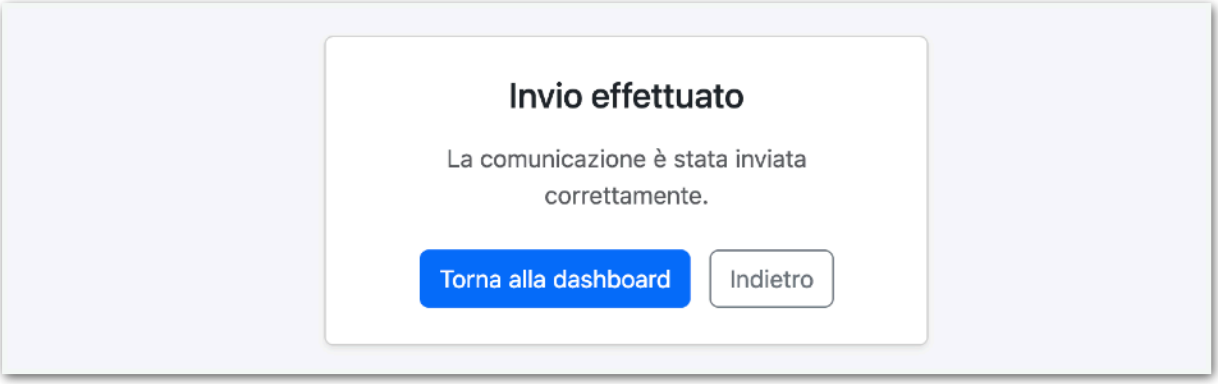

Pagamento completato con successo!
Città: Rome
Numero di biglietti: 5
Totale pagato: € 7,50
Metodo di pagamento: mastercard
Pagamento completato
QR Code biglietti
Scannerizza il QR Code per visualizzare i codici univoci dei tuoi biglietti.
1767356626246-ROME-c2b6660b
1767356626250-ROME-03c7c5f0
1767356626250-ROME-23722a39
Torna alla Home

Send Global Communication:






Confirm Communication
communication1
Back
Send Communication

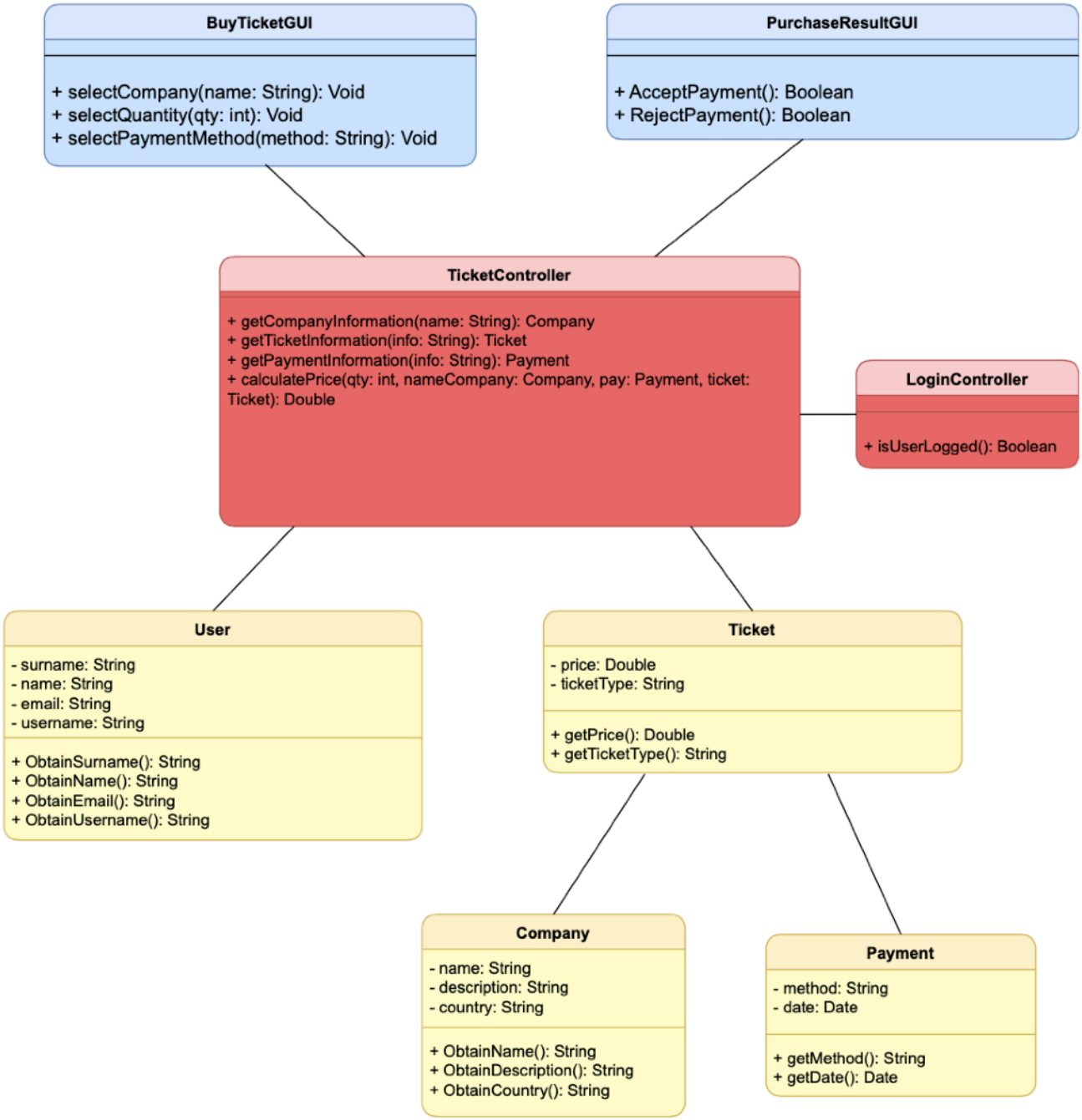

Invio effettuato
La comunicazione è stata inviata correttamente.
Torna alla dashboard
Indietro

3. DESIGN

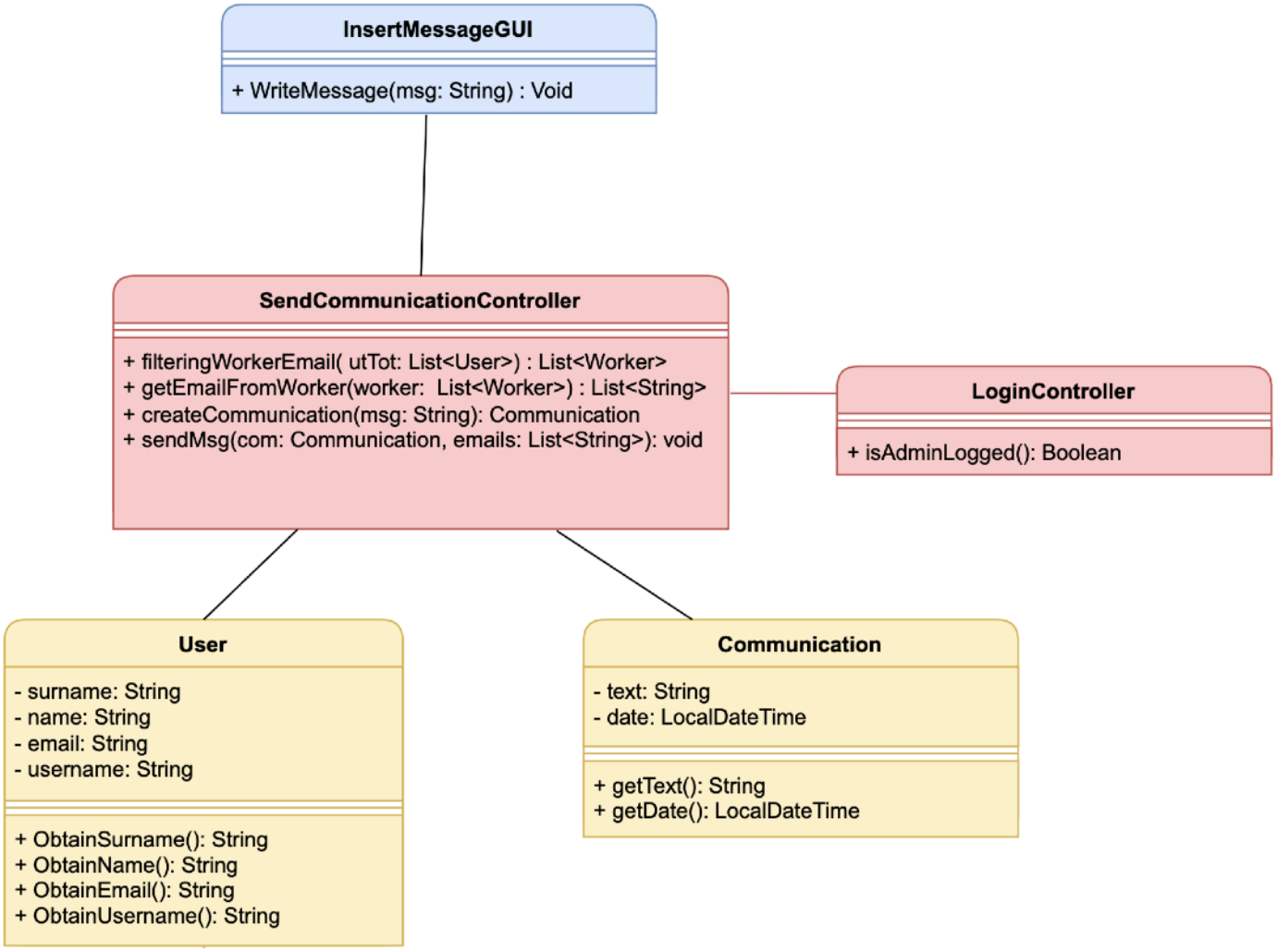
3.1 Class Diagram

3.1.1 VOPC

Buy Ticket:



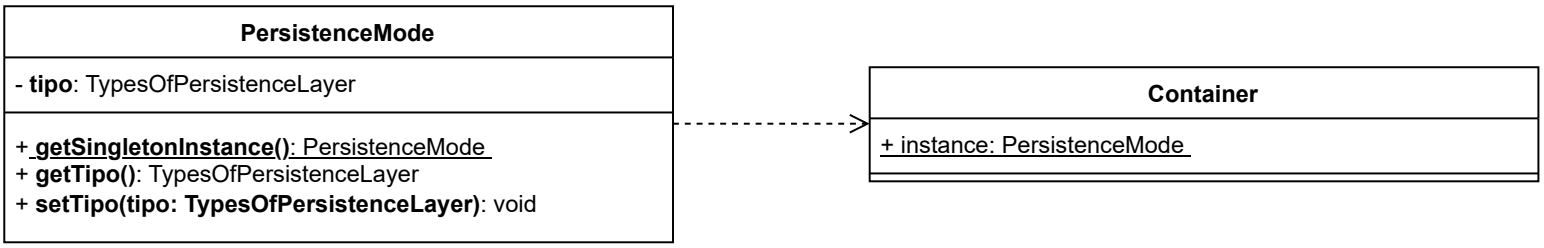
Send Global Communication:



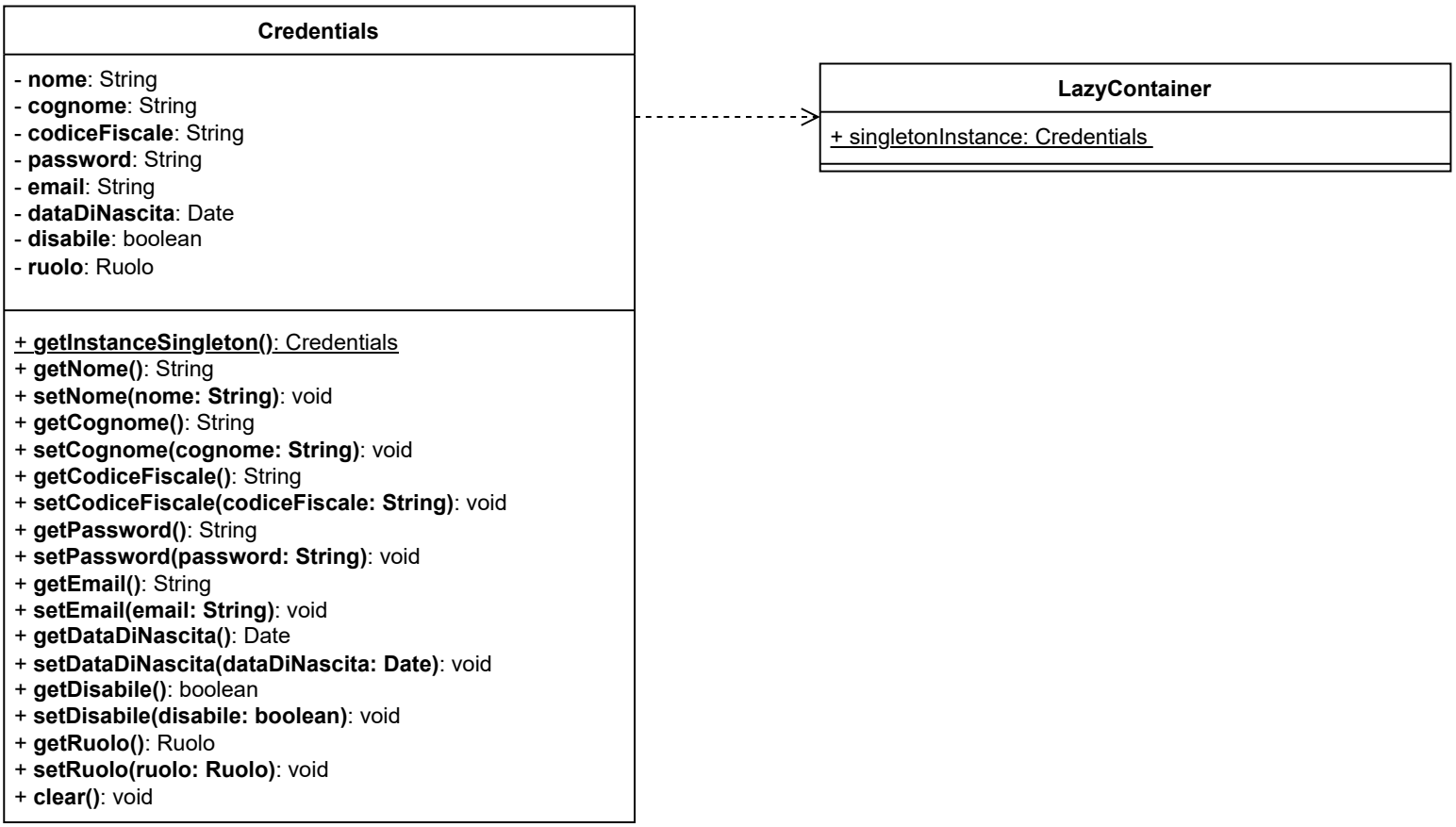
The diagram represents a simplified version of the system compared to the implemented code; since it was produced during the analysis phase, the actual implemented code (MVC) may differ from the VOPC diagrams.

3.1.2 Design-Level Diagram

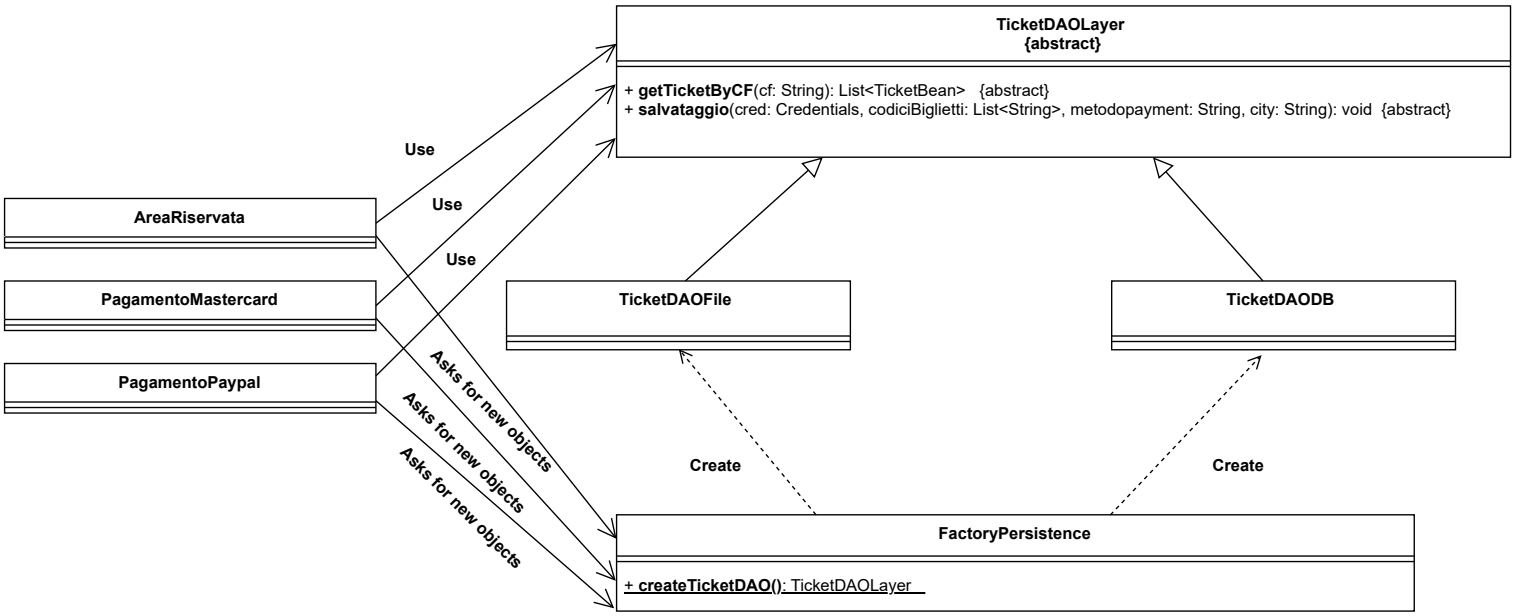
Singleton 1:



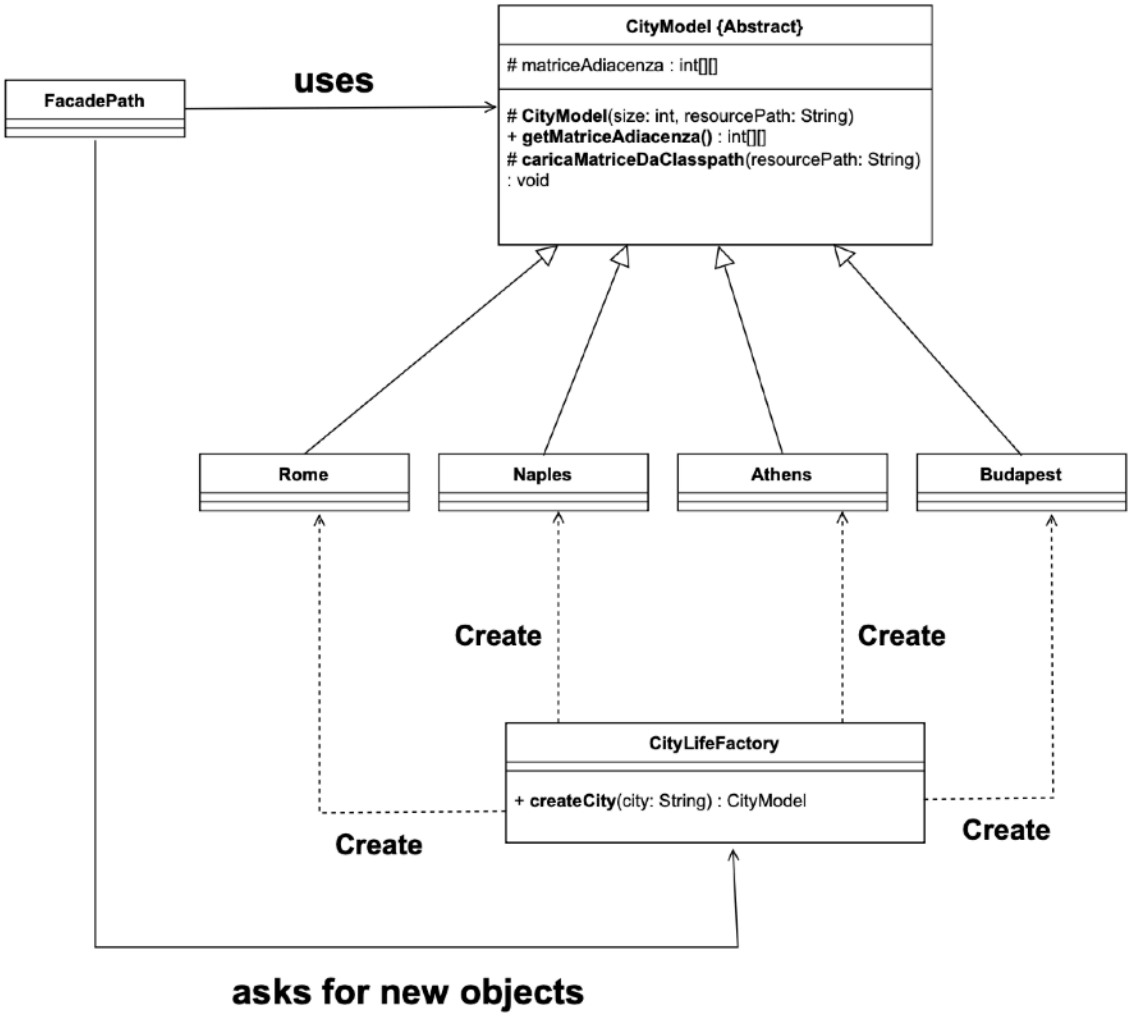
Singleton 2:



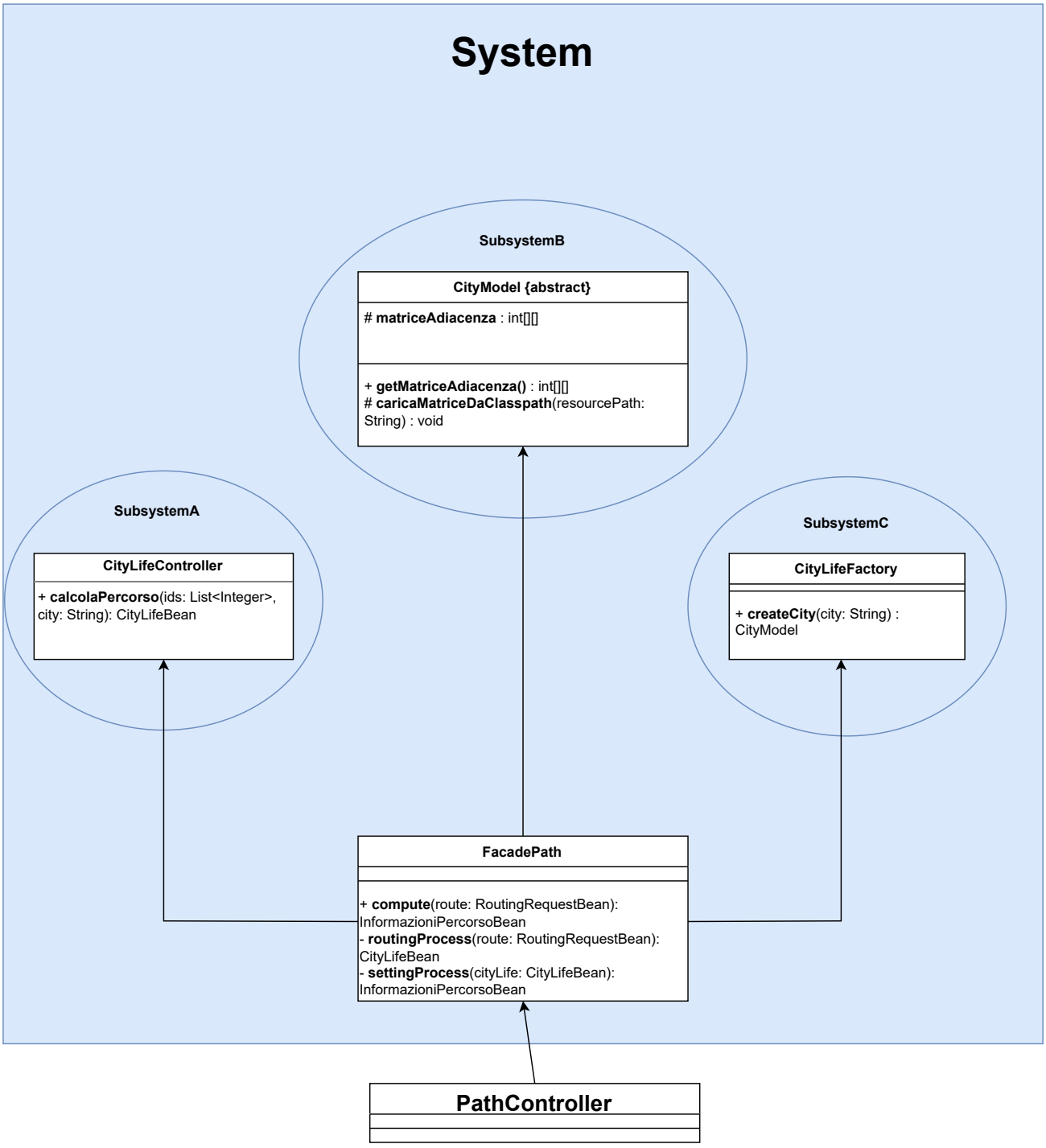
Factory Method 1 :



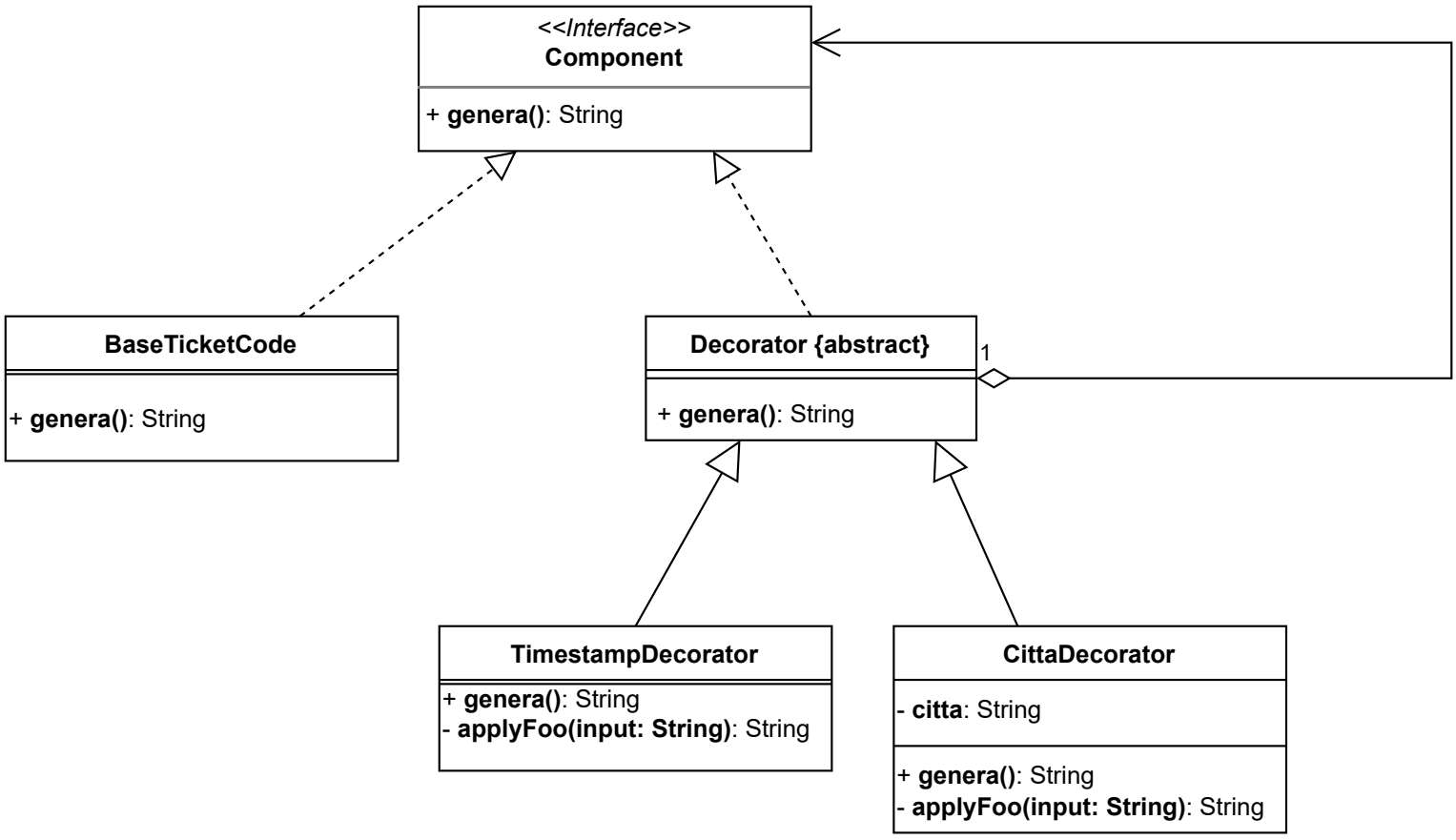
Factory Method 2 :



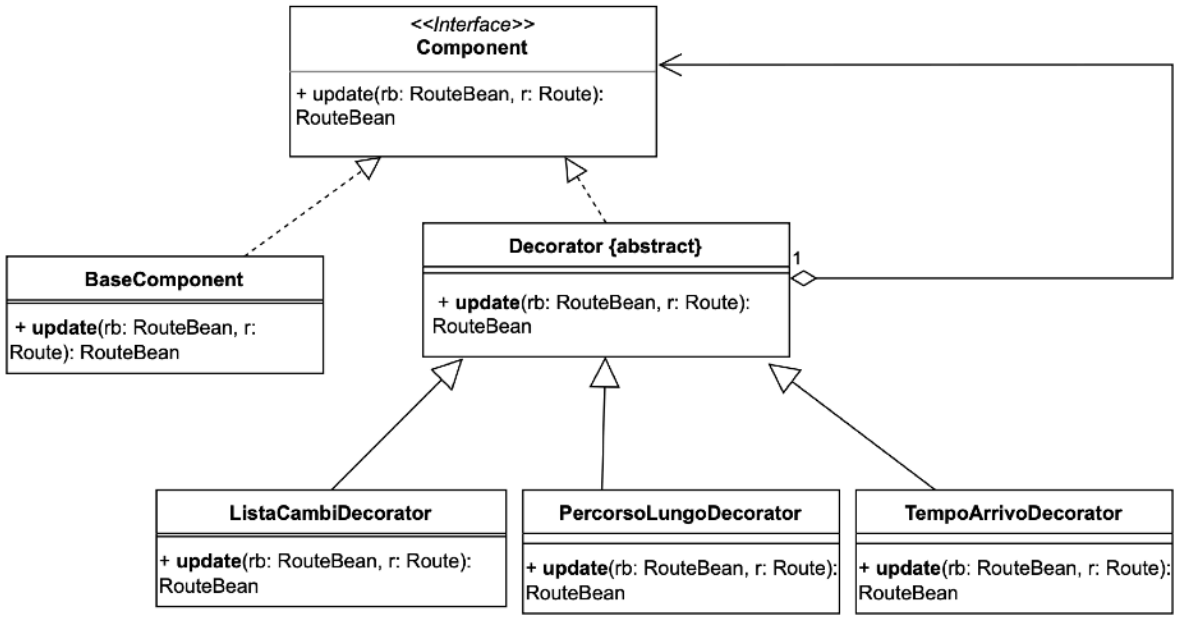
Facade:


System
SubsystemB
CityModel {abstract}
matriceAdiacenza : int[][]
+ getMatriceAdiacenza() : int[][]
# caricaMatriceDaClasspath(resourcePath: String) : void
SubsystemA
CityLifeController
+ calcolaPercorso(ids: List<Integer>, city: String): CityLifeBean
SubsystemC
CityLifeFactory
+ createCity(city: String) : CityModel
FacadePath
+ compute(route: RoutingRequestBean): InformazioniPercorsoBean
- routingProcess(route: RoutingRequestBean): CityLifeBean
- settingProcess(cityLife: CityLifeBean): InformazioniPercorsoBean
PathController

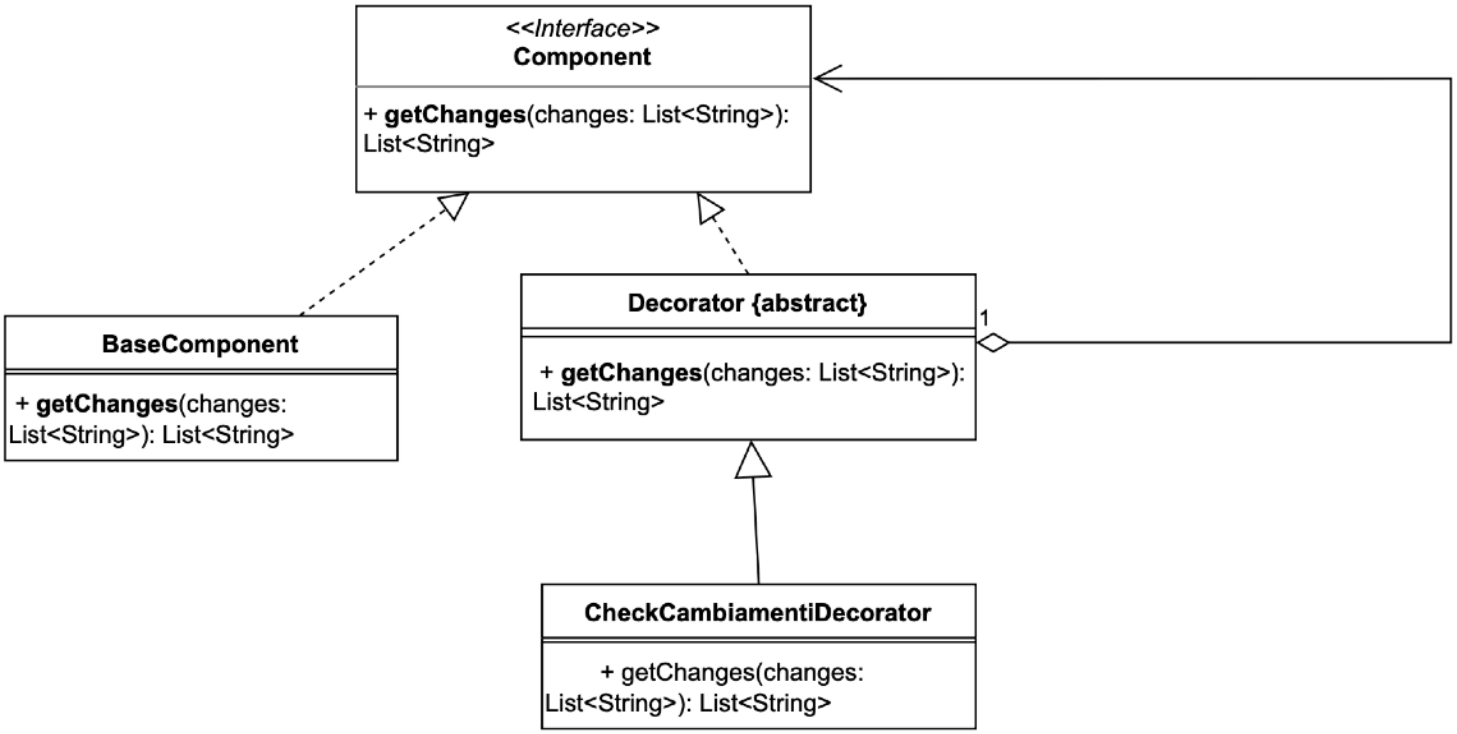
Decorator 1:



Decorator 2:



Mini Decorator 3:



3.2 Design Patterns

Factory Method Pattern:

In RouteX, we applied the Factory Method pattern to manage the creation of different city instances.
At runtime, it is possible to instantiate a specific city class, since each city provides its own adjacency matrix.
This approach represents an elegant solution, as it allows the system to create the required city only when needed.
Moreover, it significantly improves the maintainability of the software: adding a new city to the application simply requires extending the pattern with a new concrete class, without affecting the existing code.

An application of the pattern was also used to manage the dual persistence layer.
The DAO that interacts with the persistence layer is created through a dedicated factory.
In this way, also by means of an internal Singleton within the factory, it is possible to switch the persistence level at runtime, choosing between file-based and CSV-based persistence, without impacting the rest of the application.

Singleton Pattern:

In RouteX, we adopted the Singleton pattern to manage the currently logged-in user session.
This approach allows the application to maintain a single, globally accessible instance representing the active user, ensuring consistency of session data across different components.

The Singleton pattern was also applied for another purpose, namely to store the currently selected persistence level. This allows the application to maintain a single, consistent configuration regarding the active persistence mechanism, which can be accessed and shared across all components. In this context, the DAO accesses a singleton instance that stores the currently selected persistence mechanism.

Facade Pattern:

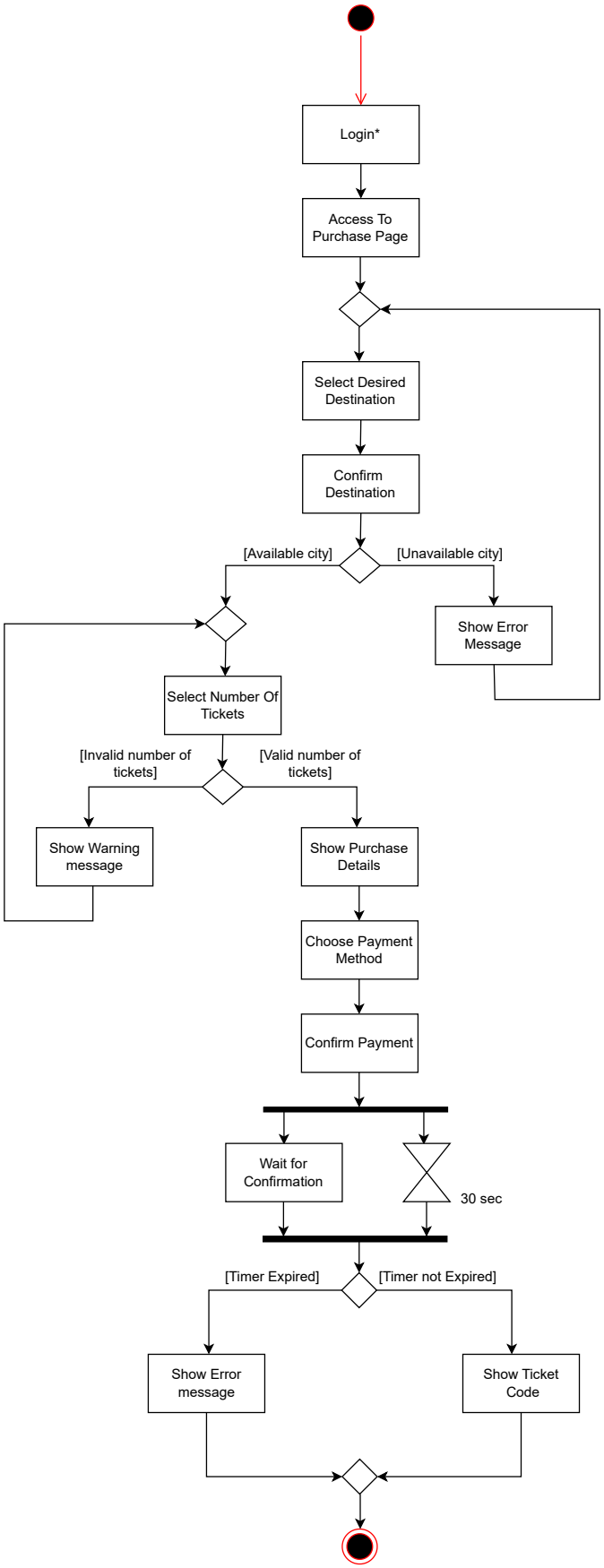
In RouteX, we applied the Facade pattern to provide a simplified interface over multiple subsystems of the application.
This pattern was used when it was necessary to explicitly execute the core functionality of the system, namely the computation of the shortest path and its related processing and storage.
By encapsulating these operations behind a single access point, the Facade reduces coupling between components and simplifies the interaction with the underlying subsystems.

Decorator Pattern:

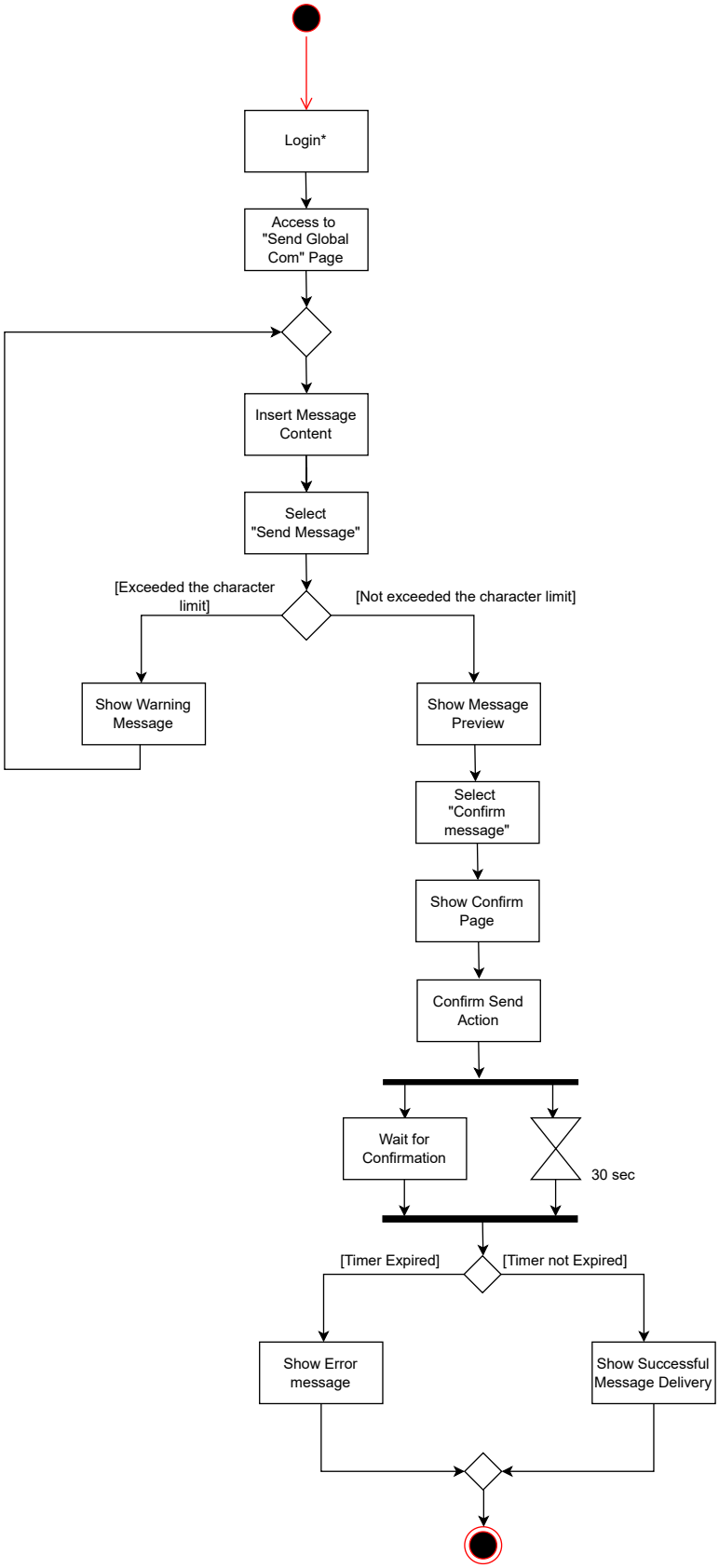
In RouteX, we applied the Decorator pattern.
This pattern works through a form of structural recursion, allowing us to add an additional aesthetic layer to the generation of the ticket code. However, its use was not limited to the graphical aspect: the Decorator was also applied at the application-logic level, where it was used to perform data validation and consistency checks.

3.3 Activity Diagram

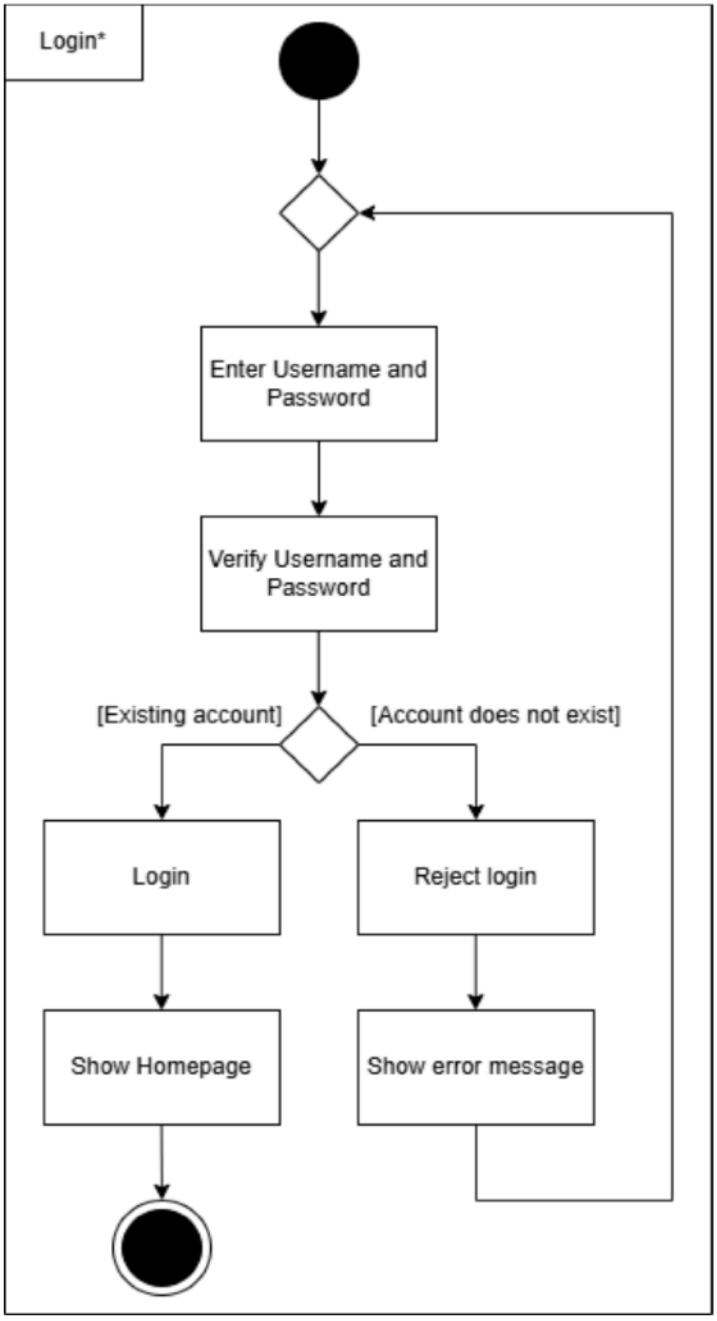
UC: Buy Ticket



UC: Send Global Communication

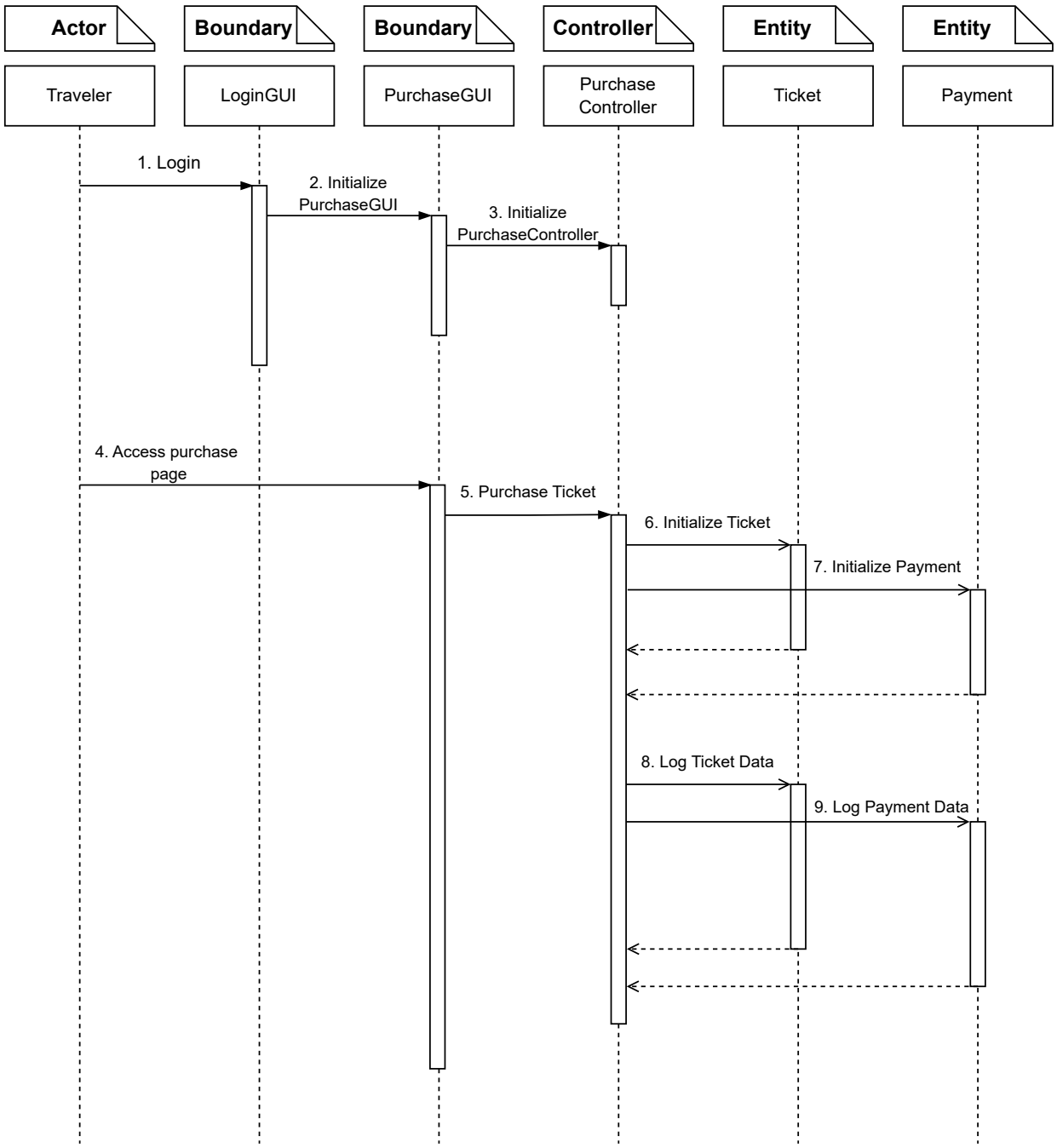


UC: Login

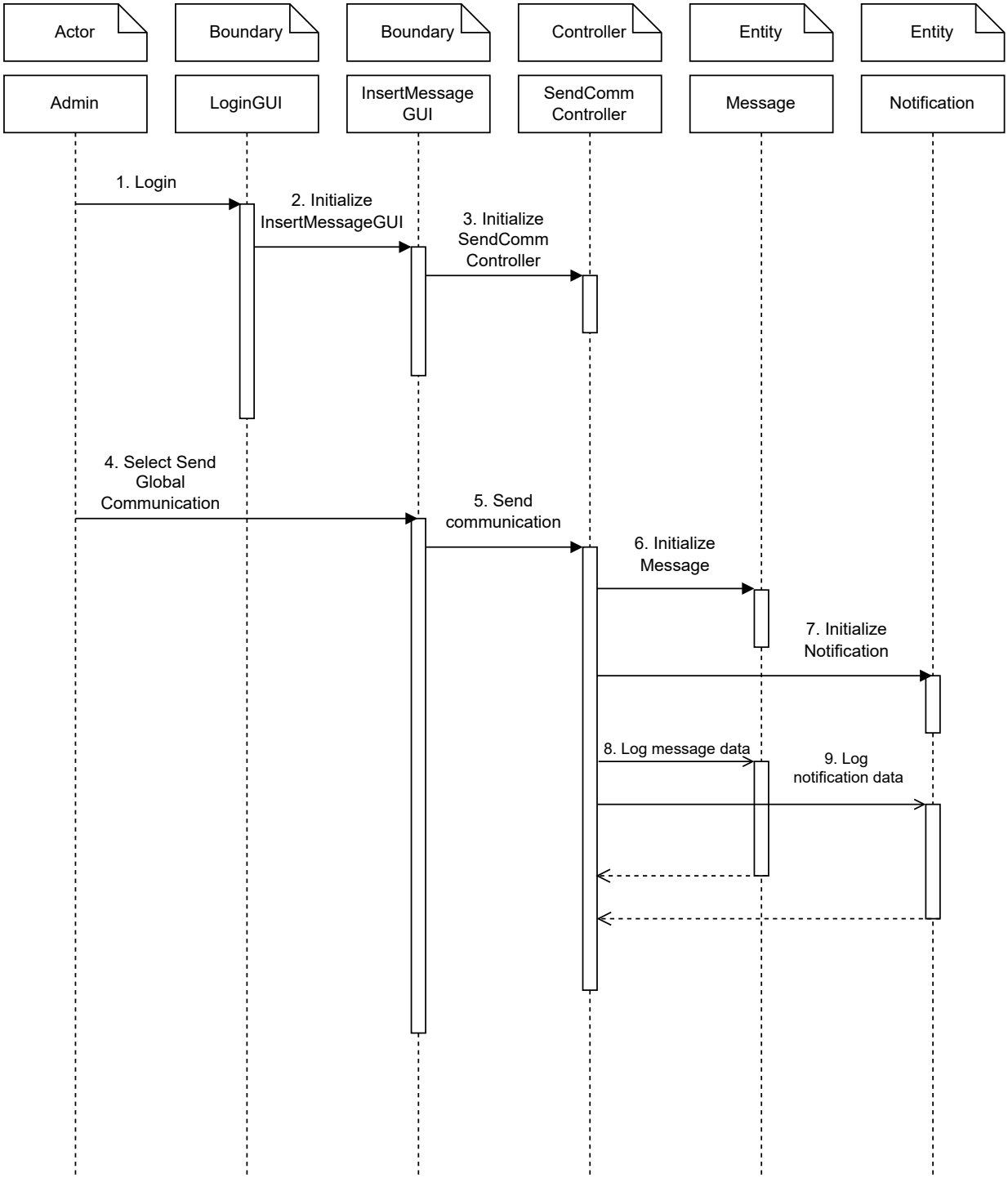

Login*
Enter Username and Password
Verify Username and Password
[Existing account]
[Account does not exist]
Login
Reject login
Show Homepage
Show error message

3.4 Sequence Diagram

UC: Buy Ticket

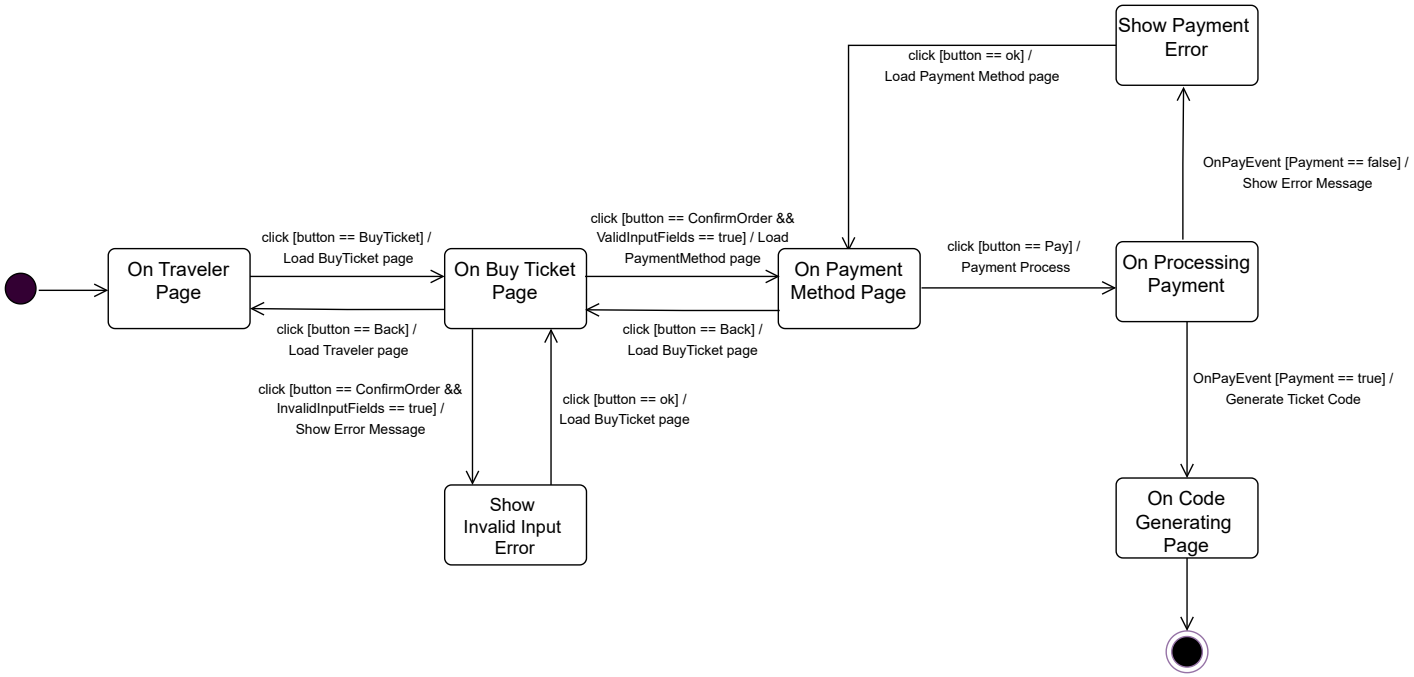


UC: Send Global Communication

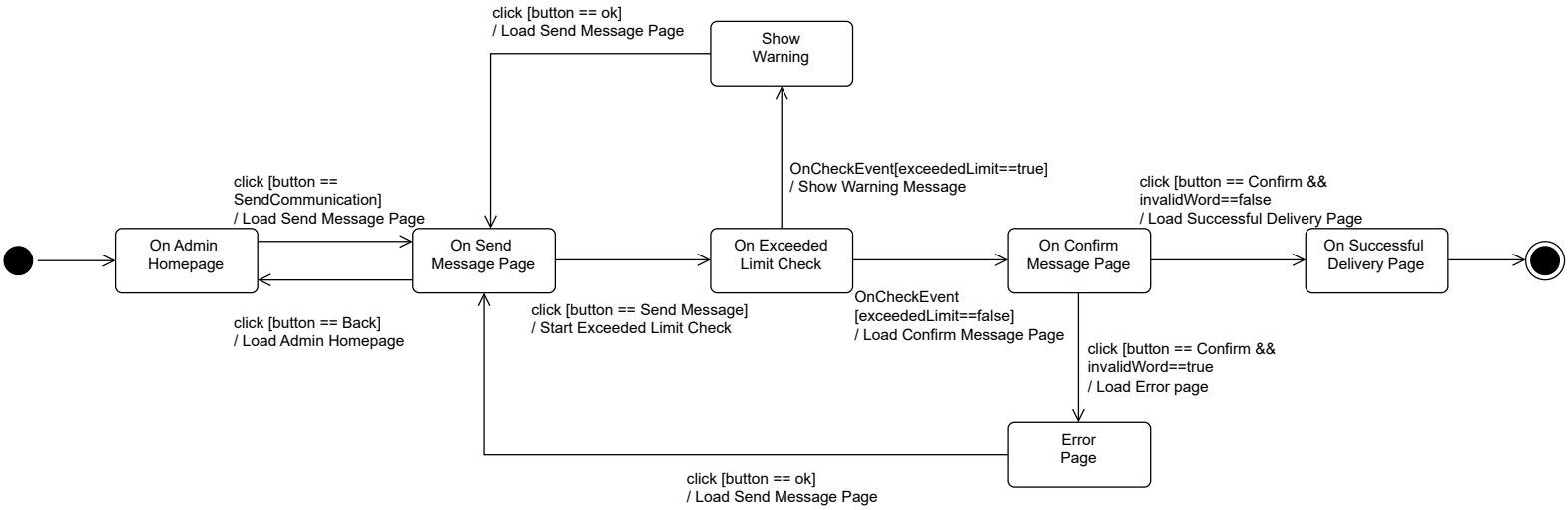


3.5 State Diagram

UC: Buy Ticket



UC: Send global Communication



4. TESTING

Testing Activities – Lorenzo Brondi

Access to reserved area data – Lorenzo Brondi

We tested the **reserved area controller** by verifying that a traveler can correctly retrieve both tickets and previously calculated routes using a valid fiscal code.
The tests ensure that the system correctly returns non-null data structures and that access to reserved information is properly handled for authenticated users.

Global communication sending – Lorenzo Brondi

We tested the communication subsystem responsible for sending global messages to users.
The tests verify that an administrator can successfully submit a message to the system and that the communication process completes correctly, returning a successful outcome after insertion.

Notification status update – Lorenzo Brondi

We tested the functionality responsible for updating the status of notifications, specifically marking them as resolved.
The tests verify that the system correctly handles valid inputs without throwing exceptions and that it safely manages edge cases such as null input arrays, ensuring robustness and fault tolerance.

Visualization of notifications – Lorenzo Brondi

We tested the notification visualization subsystem to verify that an administrator can retrieve the list of existing messages.
The tests ensure that the returned list is not null and that message contents are correctly populated when notifications are available in the system.

Worker schedule retrieval – Lorenzo Brondi

We tested the functionality that allows a worker to retrieve their work schedule.
The tests verify that, given a valid worker identifier and role, the system correctly returns the corresponding schedule without raising exceptions, ensuring proper role-based access control.

Registration workflow validation – Lorenzo Brondi

We tested the user registration process to ensure that new users can be correctly registered within the system and that the workflow executes without errors, validating the integrity of the registration subsystem.

Testing Activities – Simone Remoli

Authentication validation – Simone Remoli

We tested the authentication mechanism by verifying both valid and invalid login credentials. The tests ensure that users with incorrect credentials are correctly rejected, while valid users are successfully authenticated and provided with a proper user session.

Payment authorization and permission control – Simone Remoli

We tested the payment subsystems for both **PayPal** and **Mastercard** transactions.
These tests verify that payments cannot be executed without proper database permissions and that unauthorized users are prevented from performing payment operations.
Additionally, we validated that payments are correctly processed when the user has the appropriate role and credentials.

Prevention of payments without authenticated users – Simone Remoli

We verified that payment operations cannot be performed if no user is currently logged in. This ensures that all financial transactions are strictly bound to an authenticated session, enforcing security and consistency within the system.

Persistence layer selection and consistency – Simone Remoli

We tested the correct behavior of the persistence layer selection mechanism.
By using a factory combined with a singleton configuration object, the system dynamically switches between different persistence implementations (database or file-based).
The tests verify that the correct DAO implementation is instantiated according to the selected persistence mode.

Ticket code generation integrity – Simone Remoli

We tested the ticket code generation process, which is implemented using the Decorator pattern.
The tests verify that the generated ticket codes correctly include city-related information and timestamps, ensuring both uniqueness and correctness of the generated codes.

Access to reserved area tickets – Simone Remoli

We verified that a user can correctly retrieve their tickets when providing a valid identifier, while invalid identifiers result in appropriate exceptions.
This ensures controlled access to reserved data and prevents unauthorized retrieval of ticket information.

Route calculation and path validation – Simone Remoli

We performed integration tests on the path calculation process, verifying that valid routes correctly produce path information and related metrics.
The tests also ensure that invalid routes are properly handled by raising the appropriate exceptions, maintaining the robustness of the routing subsystem.

Access to user-specific routes – Simone Remoli

We tested that authenticated users can retrieve their previously calculated routes, while ensuring that route data is correctly associated with the requesting user.

City management and pricing validation – Simone Remoli

We tested the city management subsystem by verifying that all available cities can be correctly retrieved from the system.
Additionally, we validated the correctness of total price calculations based on the selected city and ticket quantity, ensuring consistency between business logic and returned data.

Both **Simone Remoli** and **Lorenzo Brondi** organized their tests using a **JUnit test suite**.

5. GitHub Repository

https://github.com/SimoneRemoli/RouteX.git

6. SonarCloud

https://sonarcloud.io/project/overview?id=SimoneRemoli_RouteX2

7. Presentation Video

https://www.youtube.com/watch?v=Ta4z31gq7WE